

HaqDar — Complete Build (All Files)

Run Order

1. `mkdir haqdar && cd haqdar`
2. Copy backend/ structure + paste all files
3. `pip install -r requirements.txt`
4. Set GEMINI_API_KEY in .env
5. `cd backend && python download_pdfs.py`
6. `python ingest/run_ingestion.py` ← wait for this to finish
7. `python app.py` ← keep running
8. New terminal: `cd frontend`
9. `npm create vite@latest . -- --template react`
10. `npm install && npm install -D tailwindcss postcss autoprefixer`
11. `npx tailwindcss init -p`
12. Paste all frontend files, then: `npm run dev`

FILE: backend/requirements.txt

`flask==3.0.0`

`flask-cors==4.0.0`

`python-dotenv==1.0.0`

`pypdf==4.3.1`

chromadb==0.5.0

google-generativeai==0.7.2

langchain-text-splitters==0.2.4

tenacity==8.2.3

requests==2.31.0

Pillow==10.4.0

FILE: backend/.env

GEMINI_API_KEY=your_key_here

CHROMA_DB_PATH=/absolute/path/to/haqdar/backend/chroma_db

⚠ Use absolute path for CHROMA_DB_PATH — relative paths break on restart.

FILE: backend/download_pdfs.py

"""Run once: python download_pdfs.py — downloads all legal PDFs."""

import requests

import os

DOCS = [

(

"ppc_1860.pdf",

"https://www.fmu.gov.pk/docs/laws/Pakistan%20Penal%20Code.pdf",

"https://www.ma-law.org.pk/pdflaw/PAKISTAN%20PENAL%20CODE.pdf",

),

(

"crpc_1898.pdf",

"https://www.fmu.gov.pk/docs/laws/Code_of_criminal_procedure_1898.pdf",

"https://pakistancode.gov.pk/pdf/files/administrator42956bfa0c49fd1146daa6d1a5c9cd30.pdf",

),

(

"consumer_sindh_2014.pdf",

"https://sindhlaws.gov.pk/setup/publications_SindhCode/PUB-15-000096.pdf",

None,

),

(

"tenancy_1950.pdf",

"https://www.sindhlaws.gov.pk/setup/publications/PUB-14-000123.pdf",

None,

),

(

"tenancy_1979.pdf",

"https://www.sindhlaws.gov.pk/setup/publications_SindhCode/PUB-15-000484.pdf",

None,

),

]

```
os.makedirs("documents", exist_ok=True)
```

```
HEADERS = {
```

```
    "User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36"
```

```
}
```

```
for filename, primary, backup in DOCS:
```

```
    path = os.path.join("documents", filename)
```

```
    if os.path.exists(path):
```

```
        print(f"✓ Already exists: {filename}")
```

```
        continue
```

```
    downloaded = False
```

```
    for url in filter(None, [primary, backup]):
```

```
        try:
```

```
            print(f" Downloading {filename} from {url[:60]}...")
```

```
            r = requests.get(url, headers=HEADERS, timeout=45)
```

```
            if r.status_code == 200 and len(r.content) > 5000:
```

```
                with open(path, "wb") as f:
```

```
                    f.write(r.content)
```

```
                print(f" ✓ {filename} ({len(r.content)//1024}KB)")
```

```
                downloaded = True
```

```
                break
```

```
            else:
```

```
        print(f" ✗ HTTP {r.status_code} or too small")

    except Exception as e:

        print(f" ✗ Error: {e}")

    if not downloaded:

        print(f" ✗ FAILED: {filename} — download manually and place in backend/documents/")

    print("\n=== For NIRC Labour Act: visit https://nirc.gov.pk/en/laws ===")

    print("    Download and save as: backend/documents/labour_nirc.pdf")
```

FILE: backend/app.py

```
from flask import Flask

from flask_cors import CORS

from api.routes import api_bp

import os

from dotenv import load_dotenv

load_dotenv()

app = Flask(__name__)

CORS(app, origins="*") # open for hackathon

app.register_blueprint(api_bp, url_prefix="/api")


@app.route("/health")

def health():

    return {"status": "ok", "service": "HaqDar"}
```

```
if __name__ == "__main__":
```

```
    app.run(port=5000, debug=True, threaded=True) # threaded=True — don't remove
```

FILE: backend/ingest/**init.py**

FILE: backend/rag/**init.py**

FILE: backend/api/**init.py**

FILE: backend/ingest/loader.py

```
import os
```

```
import re
```

```
import pypdf
```

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

```
# OCR fallback — install: pip install pytesseract pdf2image
```

```
# Also: brew install tesseract poppler (macOS)
```

```
try:
```

```
    import pytesseract
```

```
    from PIL import Image
```

```
    from pdf2image import convert_from_path
```

```
    OCR_AVAILABLE = True
```

```
except ImportError:
```

```
    OCR_AVAILABLE = False
```

```
    print("⚠ OCR not available (pytesseract/pdf2image not installed). Scanned PDFs will be  
skipped.")
```

```
DOCUMENTS_DIR = os.path.join(os.path.dirname(__file__), "..", "documents")
```

```
DOC_META = {
```

```
    "ppc_1860.pdf":      {"law": "Pakistan Penal Code 1860",      "doc_type": "criminal"},
```

```
    "crpc_1898.pdf":     {"law": "Code of Criminal Procedure 1898", "doc_type": "procedure"},
```

```
    "consumer_sindh_2014.pdf": {"law": "Sindh Consumer Protection Act 2014", "doc_type":  
"consumer"},
```

```
    "tenancy_1950.pdf":   {"law": "Sindh Tenancy Act 1950",      "doc_type": "tenancy"},
```

```
    "tenancy_1979.pdf":   {"law": "Sindh Rented Premises Ordinance 1979", "doc_type":  
"tenancy"},
```

```
    "labour_nirc.pdf":    {"law": "NIRC Labour Act 2010",        "doc_type": "labour"},
```

```
}
```

```
# PPC gets 800-char chunks so section citations don't cut mid-definition
```

```
PPC_SPLITTER = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=100)
```

```
DEFAULT_SPLITTER = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
```

```
def clean_text(text: str) -> str:
```

```
    text = re.sub(r"\n{3,}", "\n\n", text)
```

```
    text = re.sub(r"[\t]+", " ", text)
```

```
    text = re.sub(r"\x00", "", text)
```

```
return text.strip()
```

```
def extract_section_ref(text: str) -> str:
```

```
    """Pull out e.g. 'Section 302' or 'S. 15A' from a chunk."""
```

```
    match = re.search(r"[Ss]ection\s+(\d+[A-Z]?)[Ss]\.\s*(\d+[A-Z]?)", text)
```

```
    if match:
```

```
        return match.group(1) or match.group(2)
```

```
    return ""
```

```
def extract_text_from_pdf(pdf_path: str) -> str:
```

```
    """Extract text with OCR fallback for scanned PDFs."""
```

```
    text = ""
```

```
    try:
```

```
        reader = pypdf.PdfReader(pdf_path)
```

```
        for page in reader.pages:
```

```
            text += (page.extract_text() or "") + "\n"
```

```
    except Exception as e:
```

```
        print(f" pypdf error: {e}")
```

```
    # Scanned PDF check — if text too short, try OCR
```

```
    if len(text.strip()) < 300:
```

```
        if OCR_AVAILABLE:
```

```
            print(f" Scanned PDF detected — running OCR (this takes a while)...")
```

```
            try:
```



```

images = convert_from_path(pdf_path, dpi=200)

ocr_text = ""

for i, img in enumerate(images):

    ocr_text += pytesseract.image_to_string(img, lang="eng") + "\n"

    if i % 10 == 0:

        print(f"  OCR: page {i+1}/{len(images)}")

    if len(ocr_text.strip()) > len(text.strip()):

        text = ocr_text

except Exception as e:

    print(f"  OCR failed: {e}")

else:

    print(f"  △ Scanned PDF with no OCR fallback — text will be minimal")

return text

```

```

def load_documents() -> list[dict]:

    all_chunks = []

    for filename, meta in DOC_META.items():

        pdf_path = os.path.join(DOCUMENTS_DIR, filename)

        if not os.path.exists(pdf_path):

            print(f"△ MISSING: {filename} — skipping")

            continue

        print(f"\nLoading: {filename}")

```

```
raw = extract_text_from_pdf(pdf_path)
```

```
cleaned = clean_text(raw)
```

```
if not cleaned:
```

```
    print(f" ❌ No text extracted from {filename}")
```

```
    continue
```

```
splitter = PPC_SPLITTER if "ppc" in filename else DEFAULT_SPLITTER
```

```
chunks = splitter.split_text(cleaned)
```

```
count = 0
```

```
for i, chunk in enumerate(chunks):
```

```
    if len(chunk.strip()) < 50:
```

```
        continue
```

```
    all_chunks.append({
```

```
        "text": chunk,
```

```
        "law": meta["law"],
```

```
        "doc_type": meta["doc_type"],
```

```
        "filename": filename,
```

```
        "doc_id": f"{filename.replace('.pdf', '')}_{i}",
```

```
        "section_ref": extract_section_ref(chunk),
```

```
    })
```

```
    count += 1
```

```
print(f" ✅ {count} chunks")
```

```
print(f"\n=== Total: {len(all_chunks)} chunks loaded ===")

return all_chunks
```

FILE: backend/ingest/run_ingestion.py

```
import sys

import os

import time

sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))

from ingest.loader import load_documents

from rag.vector_store import build_vector_store

if __name__ == "__main__":

    print("=" * 50)

    print("HaqDar — Ingestion Pipeline")

    print("=" * 50)

    t0 = time.time()

    print("\n[1/2] Loading and chunking documents...")

    chunks = load_documents()

    if not chunks:

        print("\n❌ No chunks loaded. Add PDFs to backend/documents/ and retry.")

        sys.exit(1)

    print(f"\n[2/2] Embedding and storing in ChromaDB...")

    build_vector_store(chunks)
```

```
print(f"\n✓ Done in {time.time() - t0:.0f}s")
```

FILE: backend/rag/vector_store.py

```
import os

import time

import chromadb

import google.generativeai as genai

from tenacity import retry, stop_after_attempt, wait_exponential

from dotenv import load_dotenv

load_dotenv()

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

# Absolute path — relative paths break on restart

CHROMA_PATH = os.path.abspath(

    os.getenv("CHROMA_DB_PATH", os.path.join(os.path.dirname(__file__), "..", "chroma_db"))

)

def get_collection():

    client = chromadb.PersistentClient(path=CHROMA_PATH)

    return client.get_or_create_collection(

        name="haqdar_legal",

        metadata={"hnsw:space": "cosine"},

    )
```

```
@retry(stop=stop_after_attempt(3), wait=wait_exponential(multiplier=2, min=2, max=20))
```

```
def embed_text(text: str) -> list:
```

```
    result = genai.embed_content(

        model="models/text-embedding-004",

        content=text,

        task_type="retrieval_document",

    )

    time.sleep(0.15)

    return result["embedding"]
```

```
def build_vector_store(chunks: list[dict]):
```

```
    collection = get_collection()

    existing = set(collection.get()["ids"])

    new_chunks = [c for c in chunks if c["doc_id"] not in existing]

    skipped = len(chunks) - len(new_chunks)

    print(f"New: {len(new_chunks)} | Already stored: {skipped}")

    if not new_chunks:

        print("Nothing new to embed.")

    return
```

```
BATCH_SIZE = 20
```

```
total_batches = (len(new_chunks) + BATCH_SIZE - 1) // BATCH_SIZE
```

```

for i in range(0, len(new_chunks), BATCH_SIZE):

    batch = new_chunks[i : i + BATCH_SIZE]

    batch_num = i // BATCH_SIZE + 1

    print(f" Batch {batch_num}/{total_batches} ({len(batch)} chunks)...")

    embeddings = [embed_text(c["text"]) for c in batch]

    collection.add(

        ids=[c["doc_id"] for c in batch],

        embeddings=embeddings,

        documents=[c["text"] for c in batch],

        metadatas=[

            {

                "law": c["law"],

                "doc_type": c["doc_type"],

                "filename": c["filename"],

                "section_ref": c["section_ref"],

            }

            for c in batch

        ],

    )

    print(f"\n✓ Collection total: {collection.count()} chunks")

```

FILE: backend/rag/retriever.py

```
import os

import time

import google.generativeai as genai

from tenacity import retry, stop_after_attempt, wait_exponential

from rag.vector_store import get_collection

from dotenv import load_dotenv

load_dotenv()

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

@retry(stop=stop_after_attempt(3), wait=wait_exponential(multiplier=2, min=2, max=20))

def embed_query(text: str) -> list:

    result = genai.embed_content(

        model="models/text-embedding-004",

        content=text,

        task_type="retrieval_query",

    )

    time.sleep(0.1)

    return result["embedding"]

def retrieve(query: str, top_k: int = 5, doc_type_filter: str = None) -> list[dict]:

    collection = get_collection()
```

```

if collection.count() == 0:

    return []

query_emb = embed_query(query)

kwargs = dict(

    query_embeddings=[query_emb],

    n_results=min(top_k, collection.count()),

    include=["documents", "metadatas", "distances"],

)

# Only add where-filter if filtering — avoids ChromaDB errors on empty collections

if doc_type_filter:

    kwargs["where"] = {"doc_type": doc_type_filter}

results = collection.query(**kwargs)

chunks = []

for i in range(len(results["ids"][0])):

    score = 1 - results["distances"][0][i]

    # Threshold 0.15 — legal text needs a low bar (lots of keyword overlap)

    if score < 0.15:

        continue

    chunks.append({

        "text": results["documents"][0][i],

        "law": results["metadatas"][0][i]["law"],

        "doc_type": results["metadatas"][0][i]["doc_type"],

```



```
        "section_ref": results["metadatas"][0][i].get("section_ref", ""),
        "score": round(score, 3),
    })

    return sorted(chunks, key=lambda x: -x["score"])
```

FILE: backend/rag/llm.py

```
import os

import re

import json

import google.generativeai as genai

from dotenv import load_dotenv

load_dotenv()

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

SYSTEM_PROMPT = """You are HaqDar, a legal rights assistant for Pakistani citizens.

You have access to excerpts from Pakistani law including the Pakistan Penal Code (PPC),

Code of Criminal Procedure (CrPC), Sindh Consumer Protection Act 2014, Sindh Tenancy Act
1950,

Sindh Rented Premises Ordinance 1979, and NIRC Labour Act 2010.

Your job when given a user situation:

1. IDENTIFY the most relevant legal sections from the provided context

2. EXPLAIN each section in simple, plain language (no legal jargon)

3. STATE the user's rights clearly in bullet points
```

4. OUTLINE the key facts they should include if filing an FIR or complaint

5. WARN if the situation is urgent

OUTPUT as JSON only:

```
{  
  
  "summary": "...",  
  
  "relevant_sections": [{"law": "...", "section": "...", "explanation": "..."}],  
  
  "your_rights": ["...", "..."],  
  
  "fir_outline": {  
  
    "complainant": "Your full name and CNIC number",  
  
    "incident_date": "Date of the incident",  
  
    "location": "Exact location where incident occurred",  
  
    "accused": "Name or description of the accused party",  
  
    "incident_description": "What happened, in chronological order",  
  
    "applicable_sections": ["Section X of Y Act"]  
  
  },  
  
  "urgency_flag": false,  
  
  "language": "en"  
}
```

RULES:

- Never fabricate section numbers. Only cite sections visible in the provided context.
- Respond in Urdu if the user writes in Urdu, but keep all JSON keys in English.

- Always end summary with: "For serious matters, consult a licensed lawyer."
- Set urgency_flag to true for: illegal detention, assault, domestic violence, threat to life.
- If context does not cover the situation, say so honestly in the summary field. ""

```
def is_urdu(text: str) -> bool:
```

```
    arabic_chars = sum(1 for c in text if "\u0600" <= c <= "\u06FF")
```

```
    return arabic_chars / max(len(text), 1) > 0.2
```

```
def query_gemini(user_query: str, chunks: list[dict]) -> dict:
```

```
    language = "ur" if is_urdu(user_query) else "en"
```

```
    context = ""
```

```
    for chunk in chunks:
```

```
        sec = f" | Section: {chunk['section_ref']}" if chunk["section_ref"] else ""
```

```
        context += f"[Law: {chunk['law']}] {sec} \n {chunk['text']} \n \n"
```

```
    full_prompt = (
```

```
        f"LEGAL CONTEXT:\n{context}\n"
```

```
        f"USER SITUATION:\n{user_query}\n \n"
```

```
        f"Language: {language}\n"
```

```
        f"Respond with JSON only."
```

```
)
```

```
    model = genai.GenerativeModel(
```

```
        model_name="gemini-1.5-flash",
```

```
        system_instruction=SYSTEM_PROMPT,
```

```

)

response = model.generate_content(full_prompt)

raw = response.text.strip()

# Extract JSON block — handles markdown fences and extra text

match = re.search(r"\{.*\}", raw, re.DOTALL)

if not match:

    return _fallback(language, raw[:300])

try:

    return json.loads(match.group())

except json.JSONDecodeError:

    # Try stripping control chars

    cleaned = re.sub(r"[\x00-\x1F\x7F]", " ", match.group())

    try:

        return json.loads(cleaned)

    except Exception:

        return _fallback(language, raw[:300])

def _fallback(language: str, raw: str) -> dict:

    return {

        "summary": f"Could not parse structured response. Raw: {raw}. For serious matters, consult a licensed lawyer.",

        "relevant_sections": [],

```

```
"your_rights": [],  
  
"fir_outline": {},  
  
"urgency_flag": False,  
  
"language": language,  
  
}
```

FILE: backend/api/routes.py

```
import datetime  
  
from flask import Blueprint, request, jsonify  
  
from rag.retriever import retrieve  
  
from rag.llm import query_gemini, is_urdu  
  
api_bp = Blueprint("api", __name__)  
  
@api_bp.route("/health", methods=["GET"])  
  
def health():  
  
    return jsonify({"status": "ok", "service": "HaqDar"})  
  
@api_bp.route("/query", methods=["POST"])  
  
def query():  
  
    data = request.get_json(silent=True)  
  
    if not data or "query" not in data:  
  
        return jsonify({"error": "Missing 'query' field"}), 400  
  
    user_query = data["query"].strip()
```

```

if len(user_query) < 5:

    return jsonify({"error": "Query too short — describe your situation"}), 400

if len(user_query) > 3000:

    return jsonify({"error": "Query too long (max 3000 chars)"}), 400

chunks = retrieve(user_query, top_k=5)

if not chunks:

    return jsonify({

        "summary": "No relevant legal provisions found. Try describing your situation differently.
For serious matters, consult a licensed lawyer.",

        "relevant_sections": [],

        "your_rights": [],

        "fir_outline": {},

        "urgency_flag": False,

        "language": "ur" if is_urdu(user_query) else "en",

        "sources": [],

    })

result = query_gemini(user_query, chunks)

result["sources"] = [{"law": c["law"], "score": c["score"]} for c in chunks]

return jsonify(result)

@api_bp.route("/generate-fir", methods=["POST"])

def generate_fir():

```

```
data = request.get_json(silent=True) or {}

fir = data.get("fir_outline", {})

summary = data.get("summary", "")

sections = data.get("applicable_sections", [])

today = datetime.date.today().strftime("%B %d, %Y")

sections_text = (

    "\n".join(f" - {s}" for s in sections)

    if sections

    else " - [To be determined in consultation with police/lawyer]"

)

fir_text = f""""FIRST INFORMATION REPORT (FIR) — DRAFT ONLY
```

Generated by HaqDar Legal Assistant | {today}

COMPLAINANT:

Name / CNIC: {fir.get('complainant', '[Your full name and CNIC number]')}

INCIDENT DETAILS:

Date: {fir.get('incident_date', '[Date of incident]')}

Location: {fir.get('location', '[Exact location]')}

Accused: {fir.get('accused', '[Name or description of accused]')}

DESCRIPTION OF INCIDENT:

{fir.get('incident_description', '[Describe exactly what happened, step by step]')}

APPLICABLE LEGAL SECTIONS:

{sections_text}

SUMMARY:

{summary}

DISCLAIMER: This is an AI-generated draft for reference only.

Submit the official FIR at your nearest police station.

HaqDar does not provide legal representation.

For serious matters, consult a licensed lawyer.

return jsonify({"fir_text": fir_text.strip()})

FRONTEND SETUP COMMANDS

cd frontend

npm create vite@latest . -- --template react

npm install

npm install -D tailwindcss postcss autoprefixer

npx tailwindcss init -p

Then paste all frontend files below

npm run dev

FILE: frontend/vite.config.js

```
import { defineConfig } from 'vite'

import react from '@vitejs/plugin-react'

export default defineConfig({

  plugins: [react()],

  server: {

    proxy: {

      '/api': 'http://localhost:5000'

    }

  }

})
```

FILE: frontend/tailwind.config.js

```
export default {

  content: ['./index.html', './src/**/*.{js,jsx}'],

  theme: { extend: {} },

  plugins: [],

}
```

FILE: frontend/index.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8" />
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

```
<title>HaqDar — Know Your Rights</title>
```

```
<link rel="preconnect" href="https://fonts.googleapis.com" />
```

```
<link
```

```
href="https://fonts.googleapis.com/css2?family=Noto+Nastaliq+Urdu:wght@400;700&display=s  
wap"
```

```
rel="stylesheet"
```

```
/>
```

```
<style>
```

```
.font-urdu {
```

```
font-family: 'Noto Nastaliq Urdu', serif;
```

```
line-height: 2.2;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div id="root"></div>
```

```
<script type="module" src="/src/main.jsx"></script>
```

```
</body>
```

</html>

FILE: frontend/src/index.css

@tailwind base;

@tailwind components;

@tailwind utilities;

* { box-sizing: border-box; }

body { margin: 0; background: #030712; }

FILE: frontend/src/main.jsx

import { StrictMode } from 'react'

import { createRoot } from 'react-dom/client'

import './index.css'

import App from './App.jsx'

createRoot(document.getElementById('root')).render(

<StrictMode>

<App />

</StrictMode>

)

FILE: frontend/src/App.jsx

```
import { useState } from 'react'

import ResultCard from './components/ResultCard'

import FIRModal from './components/FIRModal'

const EXAMPLE_QUERIES = [

  'My landlord threw out my belongings without notice and changed the locks',

  'My employer has not paid my salary for 3 months and threatens to fire me',

  'A shopkeeper sold me a defective product and refuses to refund',

  'Someone is threatening me with violence if I report a crime',

]

function App() {

  const [query, setQuery] = useState("")

  const [result, setResult] = useState(null)

  const [loading, setLoading] = useState(false)

  const [error, setError] = useState("")

  const [firModal, setFirModal] = useState(false)

  const [firText, setFirText] = useState("")

  const [firLoading, setFirLoading] = useState(false)

  const isUrdu = (text) => {

    const arabic = [...text].filter(c => c >= '\u0600' && c <= '\u06FF').length

    return arabic / Math.max(text.length, 1) > 0.2
```

```
}

const rtl = result?.language === 'ur' || isUrdu(query)

const handleSubmit = async () => {

  if (!query.trim() || loading) return

  setLoading(true)

  setError("")

  setResult(null)

  try {

    const res = await fetch('/api/query', {

      method: 'POST',

      headers: { 'Content-Type': 'application/json' },

      body: JSON.stringify({ query }),

    })

    const data = await res.json()

    if (!res.ok) throw new Error(data.error || 'Server error')

    setResult(data)

  } catch (e) {

    setError(e.message)

  } finally {

    setLoading(false)

  }

}
```

```
const handleGenerateFIR = async () => {

  if (!result || firLoading) return

  setFirLoading(true)

  try {

    const res = await fetch('/api/generate-fir', {

      method: 'POST',

      headers: { 'Content-Type': 'application/json' },

      body: JSON.stringify({

        fir_outline: result.fir_outline,

        summary: result.summary,

        applicable_sections: result.fir_outline?.applicable_sections || [],

      }),

    })

    const data = await res.json()

    setFirText(data.fir_text)

    setFirModal(true)

  } catch (e) {

    setError('Failed to generate FIR draft')

  } finally {

    setFirLoading(false)

  }

}
```

```
}
```

```
return (
```

```
<div className="min-h-screen bg-gray-950 text-gray-100">
```

```
  { /* Header */ }
```

```
  <header className="border-b border-gray-800/60 px-6 py-5 sticky top-0 bg-gray-950/95 backdrop-blur z-10">
```

```
    <div className="max-w-3xl mx-auto flex items-center justify-between">
```

```
      <div className="flex items-center gap-3">
```

```
        <div className="w-9 h-9 bg-emerald-500/20 border border-emerald-500/40 rounded-lg flex items-center justify-center text-lg">
```



```
      </div>
```

```
    <div>
```

```
      <h1 className="text-xl font-bold text-white tracking-tight">HaqDar</h1>
```

```
      <p className="text-xs text-gray-500">Pakistani Legal Rights Assistant</p>
```

```
    </div>
```

```
  </div>
```

```
  <div className="hidden sm:flex gap-1.5">
```

```
    { ['PPC', 'CrPC', 'Consumer', 'Tenancy', 'Labour'].map(tag => (
```

```
      <span key={tag} className="text-xs bg-gray-800 text-gray-400 px-2 py-0.5 rounded-full border border-gray-700">
```

```
        {tag}
```

```
      </span>
```

```
    )}}
```

```
</div>
```

```
</div>
```

```
</header>
```

```
<main className="max-w-3xl mx-auto px-6 py-8 space-y-6">
```

```
  { /* Input Card */ }
```

```
  <div className="bg-gray-900/50 rounded-2xl border border-gray-800 p-5">
```

```
    <label className="block text-sm text-gray-400 mb-3 font-medium">
```

```
      Describe your situation in Urdu or English:
```

```
    </label>
```

```
    <textarea
```

```
      value={query}
```

```
      onChange={e => setQuery(e.target.value)}
```

```
      onKeyDown={e => { if (e.key === 'Enter' && e.ctrlKey) handleSubmit() }}
```

```
      placeholder="...مثال: میرے مکان مالک نے بغیر نوٹس مجھے گھر سے نکال دیا"
```

```
      className={`
```

```
        w-full bg-gray-800/80 text-gray-100 rounded-xl p-4 h-36 resize-none
```

```
        border border-gray-700 focus:border-emerald-500/70 focus:outline-none
```

```
        text-sm placeholder-gray-600 transition-colors leading-relaxed
```

```
        ${rtl ? 'rtl font-urdu text-right' : ''}
```

```
      `}
```

```
      dir={rtl ? 'rtl' : 'ltr'}
```


/>

<div className="flex items-center justify-between mt-3">

Ctrl+Enter to submit

<button

onClick={handleSubmit}

disabled={loading || !query.trim()}

className={

px-6 py-2.5 rounded-xl text-sm font-semibold transition-all

`\${loading || !query.trim()}`

? 'bg-gray-800 text-gray-500 cursor-not-allowed'

: 'bg-emerald-600 hover:bg-emerald-500 text-white shadow-lg shadow-emerald-900/40'

}

`}

>

{loading ? (

Analyzing...

): '⚖️ Know My Rights'}

```
</button>
```

```
</div>
```

```
{/* Example queries */}
```

```
{!result && !loading && (
```

```
<div className="mt-4 pt-4 border-t border-gray-800">
```

```
<p className="text-xs text-gray-600 mb-2">Try an example:</p>
```

```
<div className="flex flex-wrap gap-2">
```

```
{EXAMPLE_QUERIES.map((q, i) => (
```

```
<button
```

```
key={i}
```

```
onClick={() => setQuery(q)}
```

```
className="text-xs bg-gray-800 hover:bg-gray-700 text-gray-400 px-3 py-1.5  
rounded-lg border border-gray-700 transition-colors text-left"
```

```
>
```

```
{q.length > 55 ? q.slice(0, 55) + '...' : q}
```

```
</button>
```

```
)))
```

```
</div>
```

```
</div>
```

```
}}
```

```
</div>
```

```
{/* Loading state */}
```

```
{loading && (

  <div className="bg-gray-900/50 rounded-2xl border border-gray-800 p-6">

    <div className="flex items-start gap-4">

      <div className="w-8 h-8 border-2 border-emerald-400 border-t-transparent rounded-full animate-spin shrink-0 mt-0.5" />

      <div className="space-y-1">

        <p className="text-emerald-400 text-sm font-medium">Searching legal database...</p>

        <p className="text-gray-500 text-xs">Cross-referencing PPC, CrPC, Consumer Protection, Tenancy, and Labour Acts</p>

      </div>

    </div>

  </div>

)}

{/* Error */}

{error && (

  <div className="bg-red-950/40 border border-red-800/60 rounded-xl px-4 py-3 text-red-400 text-sm flex items-center gap-2">

    <span>⚠️</span> {error}

  </div>

)}

{/* Urgency Banner */}

{result?.urgency_flag && (
```

```
<div className="bg-red-950/60 border border-red-600/50 rounded-2xl px-5 py-4 flex
items-start gap-3">
```

```
<span className="text-2xl shrink-0">🚨</span>
```

```
<div>
```

```
<p className="text-red-400 font-bold text-sm uppercase tracking-wide">Urgent
Situation Detected</p>
```

```
<p className="text-red-300/80 text-sm mt-1">
```

```
  This may involve serious harm or illegal detention. Contact police (15) or a lawyer
  immediately.
```

```
</p>
```

```
</div>
```

```
</div>
```

```
  )}
```

```
{/* Result */}
```

```
{result && (
```

```
<div className={`space-y-4 ${rtl ? 'rtl font-urdu' : ''} dir={rtl ? 'rtl' : 'ltr'}>
```

```
<ResultCard result={result} rtl={rtl} />
```

```
{/* Action buttons */}
```

```
<div className="flex gap-3 justify-end pt-1">
```

```
<button
```

```
  onClick={handleGenerateFIR}
```

```
  disabled={firLoading}
```

```
  className="bg-blue-700 hover:bg-blue-600 disabled:opacity-50 text-white px-5
py-2.5 rounded-xl text-sm font-semibold transition-colors flex items-center gap-2"
```

```

>

{firLoading ? '⌚ Generating...' : '📄 Download FIR Draft'}

</button>

</div>

{/* Source tags */}

{result.sources?.length > 0 && (

  <div className="flex flex-wrap gap-2 pt-1">

    <span className="text-xs text-gray-600">Sources:</span>

    {result.sources.map((s, i) => (

      <span key={i} className="bg-gray-800 text-gray-400 text-xs px-3 py-1 rounded-full
border border-gray-700/50">

        {s.law} ({Math.round(s.score * 100)}%)

      </span>

    )))}

  </div>

)}

</div>

)}

{/* Disclaimer */}

<p className="text-center text-xs text-gray-700 pb-4">

  HaqDar provides general legal information only — not legal advice.

  Always consult a licensed lawyer for serious matters.

```

```

    </p>

  </main>

  {firModal && (
    <FIRModal text={firText} onClose={() => setFirModal(false)} />

  )}

</div>

)

}

export default App

```

FILE: frontend/src/components/ResultCard.jsx

```

import { useState } from 'react'

function Section({ title, icon, children, defaultOpen = false, badge = null }) {

  const [open, setOpen] = useState(defaultOpen)

  return (

    <div className="border border-gray-800 rounded-xl overflow-hidden">

      <button

        onClick={() => setOpen(!open)}

        className="w-full flex items-center justify-between px-4 py-3.5 bg-gray-900
        hover:bg-gray-800/80 transition-colors"

      >

        <div className="flex items-center gap-2">

```

```

<span>{icon}</span>

<span className="text-sm font-medium text-gray-200">{title}</span>

{badge && (

  <span className="text-xs bg-emerald-900/40 text-emerald-400 px-2 py-0.5
rounded-full border border-emerald-800/50">

    {badge}

  </span>

)}

</div>

<span className="text-gray-600 text-xs">{open ? '▲' : '▼'}</span>

</button>

{open && (

  <div className="px-4 py-4 bg-gray-900/30 border-t border-gray-800/50">

    {children}

  </div>

)}

</div>

)

}

export default function ResultCard({ result }) {

  return (

    <div className="space-y-3">

```

```
{/* Summary */}
```

```
<div className="bg-gray-900/50 rounded-xl border border-gray-800 p-5">
```

```
<p className="text-xs text-emerald-400 font-semibold uppercase tracking-widest mb-3">
```

```
  Legal Summary
```

```
</p>
```

```
<p className="text-gray-200 text-sm leading-relaxed">{result.summary}</p>
```

```
</div>
```

```
{/* Relevant Sections */}
```

```
<Section
```

```
  title="Relevant Legal Sections"
```

```
  icon="📋"
```

```
  defaultOpen={true}
```

```
  badge={result.relevant_sections?.length ? `${result.relevant_sections.length} found` : null}
```

```
>
```

```
{result.relevant_sections?.length > 0 ? (
```

```
  <div className="space-y-3">
```

```
    {result.relevant_sections.map((s, i) => (
```

```
      <div key={i} className="bg-gray-800/50 rounded-lg p-3.5 border border-gray-700/40">
```

```
        <div className="flex items-center gap-2 mb-1.5 flex-wrap">
```

```
          <span className="text-emerald-400 text-xs font-bold">{s.law}</span>
```

```
          {s.section && (
```



```

        <span className="text-gray-500 text-xs bg-gray-700/50 px-1.5 py-0.5 rounded">

            § {s.section}

        </span>

    )}

</div>

<p className="text-gray-300 text-sm leading-relaxed">{s.explanation}</p>

</div>

)}}

</div>

):(

    <p className="text-gray-500 text-sm italic">No specific sections identified in
database.</p>

    )}

</Section>

{ /* Your Rights */ }

<Section

    title="Your Rights"

    icon="⚖️"

    defaultOpen={true}

    badge={result.your_rights?.length ? `${result.your_rights.length} rights` : null}

>

    {result.your_rights?.length > 0 ? (

```

```

<ul className="space-y-2.5">

  {result.your_rights.map((right, i) => (

    <li key={i} className="flex items-start gap-3 text-sm text-gray-200">

      <span className="text-emerald-400 font-bold mt-0.5 shrink-0">✓ </span>

      <span className="leading-relaxed">{right}</span>

    </li>

  )})

</ul>

) : (

  <p className="text-gray-500 text-sm italic">No specific rights identified for this
situation.</p>

  )}

</Section>

{/* FIR Outline */}

<Section title="FIR Filing Guide" icon="📝">

  {result.fir_outline && Object.keys(result.fir_outline).length > 0 ? (

    <div className="space-y-2.5">

      {Object.entries(result.fir_outline).map(([key, val]) => {

        if (key === 'applicable_sections') {

          const secs = Array.isArray(val) ? val : [val]

          return (

            <div key={key}>

```

```
    <p className="text-gray-500 text-xs mb-1.5 uppercase tracking-wide">Applicable  
Sections:</p>
```

```
    <div className="flex flex-wrap gap-1.5">
```

```
      {secs.map((s, i) => (
```

```
        <span key={i} className="bg-emerald-900/30 text-emerald-400 text-xs px-2.5  
py-1 rounded-lg border border-emerald-800/40">
```

```
          {s}
```

```
        </span>
```

```
      )))
```

```
    </div>
```

```
  </div>
```

```
)
```

```
}
```

```
return (
```

```
  <div key={key} className="flex gap-3">
```

```
    <span className="text-gray-500 text-xs w-40 shrink-0 capitalize mt-0.5">
```

```
      {key.replace(/_/g, ' ')}:
```

```
    </span>
```

```
    <span className="text-gray-300 text-xs leading-relaxed">{val}</span>
```

```
  </div>
```

```
)
```

```
}}}
```

```
<div className="mt-3 pt-3 border-t border-gray-700/50">
```

```
<p className="text-xs text-gray-500">
```

💡 Fill in the fields above at your nearest police station. Bring your CNIC and any evidence.

```
</p>
```

```
</div>
```

```
</div>
```

```
) : (
```

<p className="text-gray-500 text-sm italic">No FIR outline available for this type of situation.</p>

```
}}
```

```
</Section>
```

```
</div>
```

```
)
```

```
}
```

FILE: frontend/src/components/FIRModal.jsx

```
import { useState } from 'react'
```

```
export default function FIRModal({ text, onClose }) {
```

```
  const [copied, setCopied] = useState(false)
```

```
  const handleDownload = () => {
```

```
    const blob = new Blob([text], { type: 'text/plain;charset=utf-8' })
```

```
    const url = URL.createObjectURL(blob)
```

```
const a = document.createElement('a')

a.href = url

a.download = 'HaqDar_FIR_Draft.txt'

document.body.appendChild(a)

a.click()

document.body.removeChild(a)

URL.revokeObjectURL(url)

}

const handleCopy = () => {

  // Clipboard API fallback for HTTP (no HTTPS at hackathon)

  try {

    navigator.clipboard.writeText(text).then(() => {

      setCopied(true)

      setTimeout(() => setCopied(false), 2000)

    })

  } catch {

    const textarea = document.createElement('textarea')

    textarea.value = text

    textarea.style.position = 'fixed'

    textarea.style.opacity = '0'

    document.body.appendChild(textarea)

    textarea.focus()

  }

}
```

```

        textarea.select()

        try { document.execCommand('copy') } catch {}

        document.body.removeChild(textarea)

        setCopied(true)

        setTimeout(() => setCopied(false), 2000)

    }

}

return (

    <div

        className="fixed inset-0 bg-black/85 z-50 flex items-center justify-center p-4"

        onClick={e => { if (e.target === e.currentTarget) onClose() }}

    >

        <div className="bg-gray-900 border border-gray-700 rounded-2xl w-full max-w-2xl
max-h-[85vh] flex flex-col shadow-2xl">

            <div className="flex items-center justify-between px-5 py-4 border-b border-gray-800">

                <div>

                    <h2 className="text-gray-100 font-semibold">  FIR Draft</h2>

                    <p className="text-xs text-gray-500 mt-0.5">Review and take to your nearest police
station</p>

                </div>

                <button

                    onClick={onClose}

```

```
      className="text-gray-500 hover:text-gray-300 w-8 h-8 flex items-center justify-center rounded-lg hover:bg-gray-800 transition-colors"
```

```
>
```

```
    ×
```

```
</button>
```

```
</div>
```

```
<div className="flex-1 overflow-y-auto p-5">
```

```
  <pre className="text-gray-300 text-xs font-mono whitespace-pre-wrap leading-6 select-all">
```

```
    {text}
```

```
</pre>
```

```
</div>
```

```
<div className="flex gap-3 px-5 py-4 border-t border-gray-800">
```

```
<button
```

```
  onClick={handleCopy}
```

```
    className="flex-1 bg-gray-800 hover:bg-gray-700 text-gray-200 py-2.5 rounded-xl text-sm transition-colors flex items-center justify-center gap-2"
```

```
>
```

```
  {copied ? '✓ Copied!' : '📋 Copy'}
```

```
</button>
```

```
<button
```

```
  onClick={handleDownload}
```

```
    className="flex-1 bg-emerald-600 hover:bg-emerald-500 text-white py-2.5 rounded-xl text-sm font-semibold transition-colors flex items-center justify-center gap-2"
```

```

>

<button>
  <img alt="Download icon" data-bbox="178 148 198 163"/> Download .txt
</button>

</div>

</div>

</div>

)
}

```

⚡ Critical Fixes Already Applied

Issue	Fix Applied
Scanned PDFs return empty text	pytesseract OCR fallback in loader.py
Gemini rate limits during embedding	tenacity exponential backoff, batch=20
Gemini returns broken JSON	<code>re.search(r'\{.*\}', raw, re.DOTALL)</code> extraction
ChromaDB threshold 0.3 too high	Lowered to 0.15 in retriever.py
Flask blocks on Gemini calls	<code>threaded=True</code> in app.py
FIR citations cut mid-definition	PPC uses 800-char chunks, others 500
ChromaDB path breaks on restart	<code>os.path.abspath()</code> in vector_store.py
CORS blocks deployed frontend	<code>CORS(app, origins="*")</code>
Clipboard API fails on HTTP	programmatic textarea fallback in FIRModal
Consumer Protection Act wrong	Sindh 2014 act used (not India 2019)

Demo Script (3-4pm)

1. Type: *"My landlord locked me out without notice and won't return my deposit"*
2. Show Sindh Tenancy Act / Rented Premises Ordinance sections
3. Show "Your Rights" bullets
4. Click **Download FIR Draft** → show the .txt file
5. Toggle to Urdu input to show bilingual support